

GUÍA DEL PROYECTO · VERSIÓN DEL 13 DE SEPTIEMBRE DE 2026

Radar Porteño

Cómo funciona, por dentro, un vigilante automático de talleres, cursos, ciclos de cine y promociones bancarias en Buenos Aires.

Escrita para leerse sin conocimientos previos de programación.
Cada concepto técnico se explica la primera vez que aparece.

Qué vas a encontrar acá

1. Qué hace el proyecto y qué problema resuelve

La idea de fondo, en una página

2. El viaje de un dato, de punta a punta

El recorrido completo: de una noticia en internet a una tarjeta en pantalla

3. Las fuentes: de dónde sale la información

RSS, APIs, y por qué hay tres tipos distintos

4. El modelo: cómo se representa una oferta

La ficha común, y el truco del "hash" para no repetir

5. El puntuador: cómo se separa el trigo de la paja

Reglas, no inteligencia artificial, y por qué

6. El almacén: la memoria del proyecto

SQLite, lápidas y migraciones

7. Los cuatro relojes de la vigencia

La parte más original del diseño

8. La configuración: por qué casi todo es un archivo de texto

La promesa central y su única excepción

9. La salida: cómo nace la página

Plantillas, zonas, tarjetas y color

10. La automatización: quién aprieta el botón

La tarea de Windows y el robot de GitHub

11. Errores que enseñaron algo

Los tropiezos reales y qué dejaron

12. Glosario y cómo seguir

Vocabulario y primeros pasos para tocarlo vos

1. Qué hace el proyecto y qué problema resuelve

Imaginá que querés enterarte de tres cosas: cuándo abren las inscripciones a los talleres gratuitos de las bibliotecas porteñas, qué ciclos de cine hay este mes, y qué promociones con cuotas sin interés tiene tu banco. Hoy eso significa revisar a mano una docena de sitios, todos los días, y acordarte de cuáles ya miraste.

Radar Porteño hace eso por vos, todos los días, y te deja el resultado en una sola página web. No inventa información: sale a buscarla a fuentes públicas, descarta lo que no sirve, recuerda lo que ya te mostró, y arma una página ordenada.

Las tres promesas del diseño

Todo el proyecto está construido alrededor de tres ideas. Si entendés estas tres, entendés el resto.

PRIMERA: NO REPETIR

Si una nota ya te la mostró ayer, no te la vuelve a anunciar como novedad hoy. Suena obvio, pero es la parte más difícil: la misma noticia llega desde cinco búsquedas distintas, con cinco direcciones web distintas.

SEGUNDA: NO MENTIR

Si el proyecto no sabe algo, no lo inventa. Nunca dice "cierra pronto" si no conoce la fecha de cierre. Cuando sí la conoce —porque la fuente la declara— lo dice con la fecha exacta. Esa distinción atraviesa todo el código.

TERCERA: AGREGAR UN TEMA NUEVO NO DEBE REQUERIR PROGRAMAR

Querer vigilar "ciclos de cine" en vez de "talleres de lectura" tiene que ser escribir unas líneas en un archivo de texto, no tocar el programa. Cada vez que esa promesa se rompió, se arregló el programa, no el archivo.

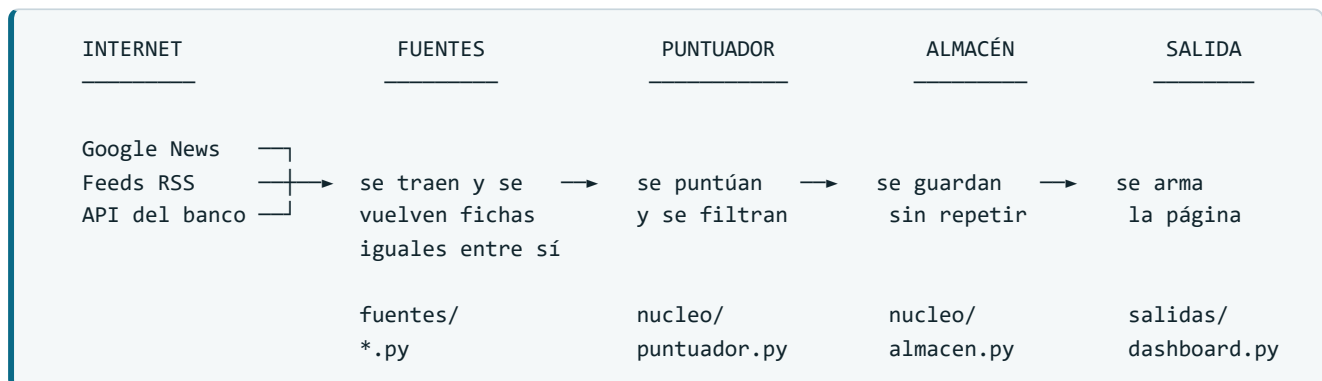
Qué vigila hoy

ZONA	QUÉ JUNTA	CÓMO SE ACTUALIZA
Ahora	Un pedido puntual: "voy a tal shopping, mostrame promos con cuotas"	A mano, cuando lo pedís
Este finde	Agenda cultural del viernes al domingo	De jueves a domingo
Cine	Ciclos, retrospectivas y muestras en la Ciudad	Todos los días
Agenda	Talleres, festivales, teatro gratis, cursos de IA e inglés	Todos los días
Ofertas	Promociones del banco y búsquedas de compra	Todos los días

El resultado se publica como una página estática en GitHub Pages: un sitio web que no necesita servidor propio ni base de datos en línea, porque es un único archivo HTML que se regenera y se sube.

2. El viaje de un dato, de punta a punta

Antes de ver las piezas por separado, conviene seguir a un dato en su recorrido completo. Tomemos una noticia real: *"La Sala Lugones presenta una retrospectiva de Frank Borzage con doce películas"*.



Paso a paso

1. **Se pregunta a las fuentes.** El programa manda seis búsquedas distintas a Google News ("ciclo de cine Buenos Aires", "Sala Lugones ciclo retrospectiva", etc.) y recibe unos 360 resultados en bruto.
2. **Todo se convierte en la misma ficha.** Venga de donde venga, cada resultado se transforma en un objeto `Oferta`: título, medio, fecha, enlace. Esto es clave: de acá en adelante, al resto del programa no le importa si vino de Google News o de una API bancaria.
3. **Se descartan los repetidos de esta misma corrida.** Las seis búsquedas devuelven la misma nota varias veces. De 360 quedan 334.
4. **Se puntúa cada una.** Se le asigna un número del 0 al 10 según qué tan bien encaja con el tema, y se anotan los motivos. Las que no cumplen los requisitos se marcan con un puntaje imposible (-99) y quedan afuera. De 334 sobreviven 9.
5. **Se guardan en la base.** Acá actúa la memoria de largo plazo: si la nota ya estaba, no se vuelve a anunciar; solo se actualiza la fecha en que se la vio por última vez.
6. **Se arma la página.** Un programa lee la base, decide qué todavía sirve mostrar, lo agrupa en zonas y familias, y escribe un archivo `index.html`.

POR QUÉ IMPORTA EL PASO 2

Convertir todo a una ficha común se llama *normalizar*. Es lo que permite que agregar una fuente nueva —mañana, la API de otro banco— no obligue a tocar el puntuador, ni el almacén, ni la página. Cada pieza habla un solo idioma con la siguiente.

3. Las fuentes: de dónde sale la información

Una "fuente" es un pedacito de programa que sabe hablar con un sitio y devolver fichas. Hay tres, y son de dos naturalezas distintas.

3.1 Qué es un RSS

Un **RSS** es una versión de un sitio web pensada para que la lean programas, no personas. En vez de colores y botones, es una lista ordenada: título, fecha, enlace, resumen. Casi todos los diarios y blogs publican uno.

Google News tiene un RSS de búsqueda: le pasás palabras y te devuelve noticias recientes de todos los medios que indexó, sin pedir registro ni clave. Eso es `fuentes/google_news.py`, y es la fuente más productiva del proyecto.

`fuentes/rss.py` hace lo mismo pero con feeds elegidos a mano, listados en `config/feeds.yaml`: el sitio del Gobierno de la Ciudad, un blog especializado en bibliotecas, agendas culturales porteñas.

3.2 Qué es una API

Una **API** es una puerta que un sitio abre para que otros programas le pidan datos directamente, en un formato estructurado. La diferencia con el RSS es que no son noticias *sobre* algo: son los datos mismos.

El caso más interesante del proyecto es `fuentes/bbva.py`. La historia vale la pena porque muestra cómo se investiga una fuente:

1. Al principio se buscaban promociones bancarias en los diarios. Se midió: de 424 noticias, la más reciente que nombraba al banco tenía **43 días**. La prensa no cubre las promos semanales.
2. Se fue a la página de beneficios del banco. Está hecha con JavaScript: el HTML llega casi vacío y el contenido aparece después.
3. Se abrió esa página en el navegador y se listaron todos los pedidos de red que hizo. Ahí apareció una dirección de la forma `go.bbva.com.ar/willgo/fgo/API/v3/communications`.
4. Se probó esa dirección desde el programa, sin navegador. **Responde**. Devuelve las promociones en formato estructurado, sin necesidad de iniciar sesión.

Esa API da algo que ninguna noticia da: **la fecha exacta hasta la que vale cada promoción**, el tope de reintegro y con qué tarjeta aplica.

UN ERROR QUE COSTÓ 280 PEDIDOS POR CORRIDA

Cuando se agregó la fuente del banco, los temas que no declaraban qué fuentes usar empezaron a correr *todas*, incluida la del banco. Cada tema de prensa se bajaba las 47 páginas del catálogo bancario para descartarlas después. Se notó porque los temas pasaron de ~300 a ~1200 resultados en bruto.

El arreglo distingue dos clases de fuente: las de *búsqueda* (reciben palabras y devuelven algo acotado) y las de *catálogo* (devuelven todo lo que tienen). Las de catálogo solo corren si un tema las pide por nombre.

3.3 El contrato

Las tres fuentes cumplen la misma regla, y es lo único que el resto del programa sabe de ellas:

```
buscar(tema, cliente) -> lista de Ofertas
```

Agregar una fuente nueva es escribir un archivo que cumpla esa firma y anotarlo en un registro. Nada más.

4. El modelo: cómo se representa una oferta

En `nucleo/modelo.py` vive la ficha común. En Python se usa una *dataclass*, que es simplemente una forma corta de describir "un objeto que tiene estos campos":

```
class Oferta:
    titulo      # "La Sala Lugones presenta una retrospectiva..."
    url         # el enlace
    fuente      # "escribiendocine.com" (el medio)
    tema        # "cine-caba"
    fecha_pub   # cuándo se publicó
    vence       # hasta cuándo vale, SI la fuente lo declara
    score       # el puntaje 0-10
    motivos     # por qué pasó el filtro
    extra       # datos propios de la fuente (tope, cuotas, logo...)
```

El truco del hash

Acá está una de las decisiones más finas del proyecto. Para saber si una oferta "ya la vimos", hace falta una identidad estable. Lo natural sería usar el enlace, pero **no funciona**: Google News entrega direcciones cifradas como `news.google.com/rss/articles/CBMi...`, y la misma nota llega con direcciones distintas desde búsquedas distintas.

La solución es fabricar una huella digital a partir del contenido:

```
identidad = huella( tema + "|" + título normalizado + "|" + medio )
```

CONCEPTO: QUÉ ES UNA "HUELLA" (HASH)

Una función de hash toma un texto de cualquier largo y devuelve siempre la misma cadena corta de letras y números. Dos textos iguales dan la misma huella; dos distintos, casi con certeza, huellas distintas. Sirve para comparar sin guardar el texto completo.

"Título normalizado" significa: en minúsculas, sin tildes, sin puntuación, y sin el sufijo " - Nombre del medio" que Google News agrega al final.

Por qué el tema forma parte de la identidad

Una misma nota puede ser relevante para dos temas a la vez: la guía del festival FIBA es a la vez "festival" y "teatro gratis". Si el tema no formara parte de la huella, el primer tema que corriera se la quedaría y en el segundo no aparecería nunca. Esto se descubrió en uso real, no en el diseño.

5. El puntuador: cómo se separa el trigo de la paja

De cada 400 resultados en bruto, sobreviven entre 3 y 25. El encargado de esa poda es `nucleo/puntuador.py`, y funciona con **reglas escritas a mano**, no con inteligencia artificial.

POR QUÉ REGLAS Y NO IA

Tres razones: cuesta cero (no hay que pagar por consulta), es *determinista* —con la misma entrada da siempre la misma salida, así que se puede depurar—, y sobre todo es **explicable**: cada oferta guarda por qué pasó el filtro. Cuando algo entra mal, se ve exactamente qué regla falló.

5.1 Los requisitos: un Y de OES

La regla principal se llama `requerir` y es una lista de grupos. **Dentro de cada grupo alcanza con que aparezca uno** de los términos; pero **todos los grupos tienen que cumplirse**. Para el tema de talleres:

```
requerir:
- [Buenos Aires, CABA, porteñ, Biblioteca Nacional] ← grupo 1: la zona
- [taller, talleres, inscripción, cupos, curso]       ← grupo 2: la actividad
- [lectura, escritura, literatura, biblioteca, libro] ← grupo 3: el tema real
```

Un solo grupo no alcanzaba, y se aprendió probando: filtrando solo por zona pasaba cualquier "agenda cultural"; filtrando solo por actividad pasaban cursos de Madrid. Y el tercer grupo se agregó porque el tema se llama "talleres de *lectura*" pero el filtro no pedía nada sobre leer: entraba el "Congreso Federal de Ciudades Inteligentes".

5.2 Las otras reglas

REGLA	QUÉ HACE	POR QUÉ EXISTE
<code>excluir</code>	Si aparece el término, se descarta	"Taller" también es galpón de reparaciones: taller mecánico, talleres del subte
<code>excluir_dominios</code>	Descarta por terminación del sitio	Medios de otros países publicando sus propias ofertas
<code>senales</code>	Suma puntos sin ser obligatorio	"Me interesa todo el rubro, pero esto va primero"
<code>max_por_dominio</code>	Tope de resultados por medio	Un sitio de baja calidad aportó 5 de 25 resultados
<code>unico_por_titulo</code>	Un mismo título, una sola vez	Las agendas se copian palabra por palabra entre medios

5.3 Las casi-repetidas

Hay un caso sutil: el mismo sitio publica "Abre la inscripción a los talleres" y "Abren las inscripciones a los talleres". Son la misma nota, pero el título cambió, así que la huella no las junta.

Para esto se compara *cuánto se parecen* dos títulos. Se toman las palabras de más de tres letras, se recortan a sus primeras cinco letras (así "inscripción" e "inscripciones" cuentan igual) y se mide qué proporción comparten. Si supera el 70%, se descarta la segunda.

LA MISMA REGLA, APLICADA MAL, ESCONDÍA OFERTAS

Ese filtro fue pensado para *titulares de prensa*. Aplicado al catálogo de un banco, donde los títulos son formularios —"Dexter 20% y 6 cuotas", "Top Sport 20% y 6 cuotas"—, colapsaba comercios distintos: ambos se reducían a {sport, cuota}. Se perdían 3 de 21 promociones.

Hoy es una opción que se apaga por tema. La lección: una regla correcta puede ser dañina en un contexto para el que no fue pensada.

6. El almacén: la memoria del proyecto

Todo lo que sobrevive al filtro se guarda en **SQLite**, que es una base de datos que vive en un solo archivo (`datos/ofertas.db`), sin servidor ni instalación. Para un proyecto de una sola persona es exactamente lo que hace falta.

6.1 Por qué la base se versiona junto al código

Normalmente una base de datos no se guarda junto al programa. Acá sí, y a propósito: el robot que corre todos los días en la nube arranca de cero cada vez. Sin la base, cada corrida "descubriría" lo mismo otra vez y te lo anunciaría como novedad todos los días.

6.2 Las lápidas

Esa decisión trae un problema: el archivo crece para siempre, y cada corrida guarda una copia entera. Por eso existe la *purga*: lo más viejo que un año se borra... pero no del todo.

Antes de borrar una fila, su huella se copia a una tabla que se llama `vistos`. Ocupa unos 40 bytes. Si esa oferta vuelve a aparecer, el programa sabe que ya la mostró y no la anuncia. Esas huellas sueltas son las **lápidas**: queda la marca de que algo existió, sin el cuerpo.

CONCEPTO: QUÉ ES UNA MIGRACIÓN

Cuando el programa necesita guardar un dato nuevo —por ejemplo, la fecha de vencimiento— hay que agregarle una columna a la base. Pero las bases que ya existen no la tienen. Una *migración* es el código que detecta esa diferencia y la corrige sola al abrir la base.

En este proyecto se llama `_migrar()` y es la razón por la que, al integrar la base del robot —que tenía el formato viejo—, las columnas nuevas aparecieron sin que nadie hiciera nada.

7. Los cuatro relojes de la vigencia

Esta es la parte más original del diseño, y vale la pena entenderla despacio. La pregunta es simple: **¿hasta cuándo sirve mostrar algo?** La respuesta resultó ser que no hay una sola, sino cuatro, y cada una nació de un problema real.

Reloj 1 — Qué entra

`max_antiguedad_dias`. Google News mezcla notas de 2018 con las de hoy. Este número decide qué tan vieja puede ser una nota para siquiera considerarla. Es un filtro de *entrada*.

Reloj 2 — Qué se sigue mostrando

`vida_util_dias`. Distinto del anterior: una nota puede haber entrado hace tres semanas y seguir siendo útil, o no. Un festival de hace 21 días ya pasó; un curso online gratuito de hace cuatro meses probablemente sigue abierto. Es un filtro de *presentación*: la fila se queda en la base, solo sale de la página.

TEMA	ENTRA HASTA	SE MUESTRA	POR QUÉ
Festivales	21 días	10 días	El evento ya pasó
Teatro gratis	35 días	30 días	Se agota o termina la temporada
Talleres	45 días	35 días	Ventana típica de inscripción
Cursos de IA	30 días	150 días	Los cursos online no cierran

Reloj 3 — La fecha que declara la fuente

Cuando el dato viene de la API del banco, no hay que estimar nada: la promoción dice que vale hasta el 27 de octubre. Ese valor se guarda en el campo `vence` y **manda sobre los dos relojes anteriores**.

Sin esto, una promoción que arrancó hace 283 días pero vence en 19 sería descartada por vieja, estando perfectamente vigente.

Reloj 4 — El calendario

La agenda del fin de semana no caduca "a los N días de publicada": caduca **el lunes**, se haya publicado el jueves o el domingo. Eso no lo puede expresar ningún reloj que cuente días hacia adelante.

Por eso existe `nucleo/calendario.py`, que calcula el próximo lunes a las 00:00 de Buenos Aires. Y algo más sutil: ese tema **solo busca de jueves a domingo**. Una agenda publicada un lunes habla del fin de semana que ya pasó; si entrara, sobreviviría hasta el lunes siguiente y la zona mostraría una semana entera de información vencida.

LA JERARQUÍA, EN UNA LÍNEA

Si la fuente declara una fecha, esa manda. Si no, mandan los dos relojes rodantes. Y si el tema caduca por calendario, esa fecha se calcula en cada corrida y se guarda como si la fuente la hubiera declarado.

8. La configuración: por qué casi todo es un archivo de texto

El archivo `config/temas.yaml` es el centro de gravedad del proyecto. **YAML** es un formato para escribir configuraciones de forma legible: nombre, dos puntos, valor; la sangría indica qué está adentro de qué.

Un tema completo se ve así (abreviado):

```
- nombre: cine-caba
  zona: Cine # en qué pestaña aparece
  familia: Ciclos y muestras # cómo se agrupa dentro de la pestaña
  etiqueta: "Ciclos de cine" # cómo se lee en pantalla
  keywords: # qué se le pregunta a Google News
    - "ciclo de cine Buenos Aires"
    - "Sala Lugones ciclo retrospectiva"
  requerir: [...] # los grupos obligatorios
  excluir: [...] # lo que descarta
  max_antiguedad_dias: 30
  vida_util_dias: 45
  score_minimo: 6.5
```

Fijate que ahí conviven dos cosas muy distintas: *qué buscar* (keywords, requerir) y *cómo mostrarlo* (zona, familia, etiqueta). Están juntas a propósito. Cuando la etiqueta vivía en el código, agregar un tema obligaba a editar dos archivos, y eso rompía la tercera promesa.

LA ÚNICA EXCEPCIÓN RECONOCIDA

Los colores de acento de cada zona (teal, ámbar, violeta, rosa, verde) están definidos en la hoja de estilos. Agregar una zona con un color nuevo *sí* obliga a tocar el CSS. Está documentado como excepción en vez de fingir que no existe.

Un detalle de YAML que causó un error real

YAML permite reutilizar una lista con un "ancla": se define una vez y se referencia desde varios temas. Pero **no concatena**: si reusás una lista y además querés agregarle términos propios, el resultado es una lista que contiene otra lista adentro.

Cuando se creó el ancla de zona compartida, se perdió sin querer el término "Biblioteca Nacional", que estaba en el grupo original. Los dos avisos de la Biblioteca Nacional —uno de ellos el mejor puntuado de todos, con 10.0— murieron filtrados por "fuera de zona". Hoy el programa aplana esas listas anidadas antes de usarlas.

9. La salida: cómo nace la página

9.1 Qué es una plantilla

La página no se escribe a mano ni se arma pegando texto. Se usa una **plantilla**: un archivo HTML normal con huecos y pequeñas instrucciones. La herramienta se llama Jinja2:

```
<h3>{{ oferta.titulo }}</h3>

{% for o in tema.ofertas %}
    ... acá se repite una tarjeta por cada oferta ...
{% endfor %}

{% if o.nuevo %}<span class="led"></span>{% endif %}
```

Las llaves dobles insertan un valor; las llaves con porcentaje son lógica: repetir, o mostrar solo si se cumple algo. El programa `salidas/dashboard.py` prepara los datos y la plantilla decide cómo se ven.

9.2 Tres niveles: zona, familia, tema

```
ZONA "Ofertas"           ← una pestaña en el panel izquierdo
└─ FAMILIA "Bancos y tarjetas" ← un subtítulo
    └─ TEMA "BBVA"           ← un acordeón desplegable
    └─ TEMA "Pelota de fútbol"
```

La separación en zonas nació de un problema concreto: una agenda cultural y una promoción bancaria *se leen distinto*. La agenda se hojea ("qué hay para hacer"); la oferta se consulta ("qué descuento tengo hoy"). Mezcladas en una lista, la promoción de hoy queda enterrada entre talleres de hace tres semanas.

9.3 Dos tipos de tarjeta

Por la misma razón hay dos diseños de tarjeta. En Agenda importa el puntaje, el medio y la antigüedad. En Ofertas importa el comercio, el beneficio, el tope y hasta cuándo vale.

Esto no fue una decisión estética sino una medición: en el tema de ofertas había **21 tarjetas con solo dos valores de puntaje distintos**. Un número grande que siempre dice lo mismo gasta espacio. En cambio, datos que decidían la compra —el tope de reintegro, qué tarjeta aplica— estaban guardados pero no se mostraban en ningún lado.

9.4 El color, y por qué está medido

Los colores no están escritos uno por uno en cada elemento, sino declarados una vez como *variables* y usados por nombre. Eso permite que cambiar una variable cambie toda la página, y que cada zona redefina su acento sin duplicar reglas.

CONCEPTO: CONTRASTE Y POR QUÉ SE MIDE

El contraste entre un texto y su fondo se expresa como una proporción: 1:1 es invisible, 21:1 es negro sobre blanco. La norma internacional de accesibilidad (WCAG) pide al menos **4.5:1** para texto normal.

En este proyecto se midió con un script, y aparecieron fallas reales que a ojo no se notaban: un gris que se usaba en "Tarjetas de crédito BBVA" y "buscar la nota" daba 3.11:1. Se corrigió moviendo solo la luminosidad, para no cambiar el tono de la paleta. Hoy se miden 37 pares en cada modo.

10. La automatización: quién aprieta el botón

El proyecto se ejecuta solo, por dos caminos paralelos.

10.1 La tarea programada de Windows

Todos los días a las 9:00, Windows ejecuta un archivo `.bat` que corre las cuatro etapas —buscar, sincronizar con Google Sheets, generar la página, publicar— y deja un registro en un archivo de texto.

Un detalle chico pero real: antes, esa tarea abría una consola negra en primer plano cada mañana. Ahora la llama a través de un pequeño archivo intermedio que la ejecuta con la ventana oculta.

POR QUÉ NO SE RESOLVIÓ DE LA FORMA OBVIA

Windows ofrece una opción "ejecutar aunque el usuario no haya iniciado sesión", que también oculta la ventana. No se usó: la publicación necesita una credencial de Git que está guardada en la sesión del usuario, y fuera de esa sesión el paso final fallaría. La solución fácil habría roto algo silenciosamente.

10.2 El robot de GitHub

En paralelo, un servicio de GitHub llamado *Actions* corre lo mismo todos los días a las 12:00 UTC (9 de la mañana en Buenos Aires), en una computadora prestada en la nube. Sirve para que el proyecto siga funcionando aunque tu máquina esté apagada.

10.3 Las credenciales

Las claves —la cuenta de servicio de Google, el identificador de la planilla— **nunca** se guardan junto al código. En la nube viven en un cofre de GitHub llamado *secrets*; el robot las escribe en disco al empezar y las borra antes de publicar. En tu máquina viven en archivos que están explícitamente excluidos.

Verificado al hacer el último commit: esas claves nunca estuvieron en el historial del repositorio.

11. Errores que enseñaron algo

Esta sección existe porque las decisiones buenas casi siempre vienen de un error concreto, y entender el error explica la decisión mejor que cualquier justificación abstracta.

El enlace que no se puede resolver

Google News cifra las direcciones reales de los medios. Se intentó descifrarlas y falla prácticamente siempre. En vez de fingir que funciona, el proyecto hace dos cosas: no depende de esos enlaces para identificar ofertas —de ahí el hash por título y medio— y ofrece un enlace de respaldo que busca el título exacto en Google, que siempre funciona.

La misma búsqueda, mucho ruido geográfico

El ruido casi nunca fue temático sino *geográfico*. "Buenos Aires" nombra a la Ciudad y a la Provincia. Los festivales traían el festival de cine de la Provincia, la universidad de Lanús, Avellaneda. Para cine, el festival de La Plata aparecía en casi todas las búsquedas. La lista de exclusiones creció midiendo, no imaginando.

Cuando un filtro se endurece, lo ya guardado no cambia

Mejorar las reglas no toca lo que ya está en la base. Después de cada cambio hay que decidir si vale la pena reevaluar lo guardado. Una vez se reevaluaron 48 filas: 9 se borraron y 6 cambiaron de puntaje.

Medir sobre algo invisible da cero

Durante una revisión, se midió la posición de un elemento y daba 0. La conclusión fue que estaba roto. No lo estaba: la medición había caído sobre una zona oculta, y en ese caso el navegador devuelve ceros para todo. La lección quedó anotada: medir siempre partiendo de lo que está visible.

Un carácter invisible que rompía una búsqueda

Al escribir código desde la terminal, ciertas secuencias como `\b` se convertían silenciosamente en caracteres de control invisibles. El programa quedaba válido, la búsqueda nunca encontraba nada, y ni siquiera se veía al inspeccionar el archivo. Pasó tres veces antes de identificarse.

12. Glosario y cómo seguir

API

Puerta que un sitio abre para que otros programas le pidan datos en formato estructurado, sin pasar por la página web.

Dataclass

En Python, una forma corta de definir un objeto que solo agrupa campos.

Dedup (deduplicación)

Detectar y descartar lo que ya se tiene, para no repetirlo.

Hash (huella)

Cadena corta calculada a partir de un texto. El mismo texto da siempre la misma huella.

Jinja2

Herramienta que rellena plantillas: un HTML con huecos y lógica simple.

JSON

Formato de texto para intercambiar datos entre programas. Es lo que devuelven casi todas las APIs.

Normalizar

Llevar textos distintos a una forma común para poder compararlos: minúsculas, sin tildes, sin puntuación.

RSS

Versión de un sitio pensada para programas: una lista de títulos, fechas y enlaces.

SQLite

Base de datos que vive en un solo archivo, sin servidor.

WCAG

Norma internacional de accesibilidad web. De ahí sale el mínimo de 4.5:1 de contraste para texto.

YAML

Formato de configuración legible por personas, donde la sangría indica jerarquía.

Los comandos que vas a usar

```
run.py buscar           busca en todos los temas y guarda
run.py buscar -t cine-caba  corre un solo tema
run.py buscar --seco       busca y muestra, SIN escribir en la base
run.py dashboard         regenera la página
run.py ahora ...          pedido puntual para la zona "Ahora"
run.py listar            muestra lo guardado
run.py purgar --dias 365  borra lo muy viejo dejando su lápida
```

EL COMANDO MÁS ÚTIL PARA APRENDER

`run.py buscar -t <tema> --seco` es la mejor forma de entender el proyecto tocándolo. Busca de verdad, imprime cada resultado con su puntaje y sus motivos, y **no escribe nada**. Podés cambiar una regla en el YAML, correrlo, y ver al instante qué entra y qué sale.

Tres cosas que podrías hacer vos

1. **Agregar un tema.** Copiá un bloque de `config/temas.yaml`, cambiale el nombre, las keywords y los requisitos. Corré con `--seco` hasta que los resultados te gusten.
2. **Ajustar un filtro existente.** Si algo se cuela, agregá el término a `excluir`. Si falta algo bueno, mirá con `--seco` por qué se descartó: el programa imprime el motivo.
3. **Sumar una fuente RSS.** Buscá el feed de un sitio (probá agregarle `/feed` o `/rss` a la dirección), verificá que devuelva algo, y anotalo en `config/feeds.yaml`.

Lo que quedó pendiente

- **Mercado Pago.** Su sitio público de promociones muestra contenido de mayo. No es un problema de acceso: la fuente está desactualizada. Las promos vigentes viven dentro de la aplicación.
- **El aviso por Telegram.** Está escrito y probado, pero apagado. Se enciende completando dos claves.
- **Un radar de verdad.** El sitio se llama Radar Porteño y nunca mostró un radar. Un gráfico polar donde cada oferta es un punto —el ángulo según la familia, la distancia al centro según la urgencia— usaría datos que hoy solo viven en tarjetas.

Fin de la guía. Está escrita sobre el estado del proyecto al 13 de septiembre de 2026; los números concretos (cuántas ofertas, cuántas páginas de catálogo) cambian con cada corrida, pero las ideas no.